

Stodoo - Documentation Développeur

1. Structure du Projet

Le projet est un monorepo contenant le backend et le frontend.

```
/
├── backend/                                # API FastAPI (Python)
│   ├── app/
│   │   ├── main.py                        # Routes API et initialisation
│   │   ├── core.py                       # Logique coeur (Stripe +
│   │   │   └── Orchestration)
│   │   ├── payplug_sync.py               # Logique spécifique PayPlug
│   │   ├── models.py                    # Schémas Pydantic
│   │   ├── security.py                   # Gestion du chiffrement AES
│   │   └── static/                       # Build du frontend (généré)
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                              # Interface React (TypeScript + Vite)
│   ├── src/
│   │   ├── components/
│   │   ├── Dashboard.tsx                 # Vue principale
│   │   └── ConfigModal.tsx               # Gestion des configurations
│   └── tailwind.config.js
└── deploy_prod.ps1                       # Script de déploiement
```

2. Backend - Logique de Synchronisation

2.1. Orchestration (core.py)

La fonction `process_sync_for_client` est le point d'entrée. Elle : 1. Déchiffre les secrets nécessaires. 2. Initialise la connexion XML-RPC avec Odoo. 3. Détermine la date de début (`last_sync_date` ou date forcée). 4. Exécute la boucle Stripe : * Récupère les `BalanceTransaction` de type `charge`, `refund`, `payout`. * Extrait les

métadonnées (ID commande) via `extract_label`. * Cherche le partenaire dans Odoo via l'email (`get_odoo_partner_id`). * Crée les lignes dans Odoo. 5. Appelle le module PayPlug.

2.2. Connecteur PayPlug (`payplug_sync.py`)

Ce module est plus complexe en raison de la pagination et des rapports de virement. * **Time-Walking Pagination** : L'API PayPlug limite à 100 pages. Pour contourner cela, le script déplace dynamiquement la fenêtre temporelle vers le passé en utilisant `created_at[lte]`. * **Accounting Reports** : Contrairement à Stripe, les virements (payouts) ne sont pas des objets de transaction simples. Le script demande la génération d'un rapport CSV à PayPlug, attend sa disponibilité, le télécharge et le parse pour extraire les lignes de type Transfer.

2.3. Sécurité (`security.py`)

Utilise la bibliothèque `cryptography.fernet`. * `encrypt(text)` : Renvoie une chaîne chiffrée. * `decrypt(encrypted_text)` : Renvoie le texte clair. * Nécessite `MASTER_ENCRYPTION_KEY` d'une longueur de 32 octets encodée en Base64.

3. Frontend - Interface de Gestion

3.1. Gestion des Secrets

Les secrets ne sont jamais affichés en clair après avoir été enregistrés. Le frontend envoie les nouvelles valeurs au backend qui les chiffre. Si le champ reste vide lors d'une modification, le backend conserve la valeur chiffrée existante.

3.2. Test de Connexion Odoo

Le frontend appelle `/api/odoo/check`. Le backend tente une authentification XML-RPC et, en cas de succès, liste les journaux de type bank. Cela permet à l'utilisateur de sélectionner les IDs corrects sans erreur de saisie.

4. Guide d'Extension : Ajouter un nouveau connecteur

Pour ajouter un nouveau PSP (ex: PayPal, Mollie) : 1. **Modèles** : Mettez à jour `backend/app/models.py` pour inclure la nouvelle configuration. 2. **Logic de Sync** : Créez un nouveau fichier `backend/app/psp_sync.py` inspiré de `payplug_sync.py`. 3. **Intégration Core** : Appelez votre nouvelle fonction dans `process_sync_for_client` dans `core.py`. 4. **Frontend** : * Mettez à jour `ConfigModal.tsx` pour ajouter les champs de formulaire. * Ajoutez un onglet dans `Dashboard.tsx`.

5. Développement Local

Prérequis

- Python 3.9+
- Node.js 18+

Backend

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
export MASTER_ENCRYPTION_KEY=votre_cle_de_32_chars_base64
uvicorn app.main:app --reload
```

Frontend

```
cd frontend
npm install
npm run dev
```

Le frontend est configuré (via `vite.config.ts`) pour proxier les appels `/api` vers `localhost:8000`.

6. Intégration Odoo (XML-RPC)

L'application utilise le protocole standard XML-RPC d'Odoo sur les points d'entrée `/xmlrpc/2/common` (auth) et `/xmlrpc/2/object` (données).

Modèles utilisés :

- `res.partner` : Recherche par email.
- `account.journal` : Lecture des journaux bancaires.
- `account.bank.statement.line` : Création des flux.
- `account.move` : Pour corriger le compte comptable des lignes de frais (`bouton_draft` -> `write` -> `action_post`).